

## Scaling Book — Section 3 Exercises

Rahul Bir

### Pop Quiz (2D sharding across 1 axis)

$A = \text{fp32}[1024, 4096]$ , sharding:  $A[I_{XY}, J]$ , mesh =  $\{X:8, Y:2\} \Rightarrow$  total 16 devices.

Each local shape:  $\text{fp32}[\frac{1024}{16}, 4096] = \text{fp32}[64, 4096]$ .

Local shard is  $4 \cdot 64 \cdot 4096$  bytes.

On H100 (bw =  $3.4\text{e}12$  bytes/s):

$$T_{\text{load}} = \frac{4 \cdot 64 \cdot 4096}{3.4\text{e}12} \text{ s} = 3 \times 10^{-7} \text{ s}.$$

### Pop Quiz Q2

$A = \text{int8}[128, 2048]$ , mesh =  $\{X:2, Y:8, Z:2\}$ , sharding:  $A[I_{XY}, J]$ .

$$\text{local shape} = \text{int8}[\frac{128}{16}, 2048] = \text{int8}[8, 2048].$$

per-device  $A$  mem usage =  $8 \times 2048$  bytes = 16,384 bytes.

across all devices =  $2 \cdot 8 \cdot 2 \cdot 16,384$  bytes = 524,288 bytes.

### Pop Quiz 2 [AllGather time]

$\text{AllGather}_Y([E_Y, F]) \rightarrow [E, F]$  on TPU v5e. mesh =  $\{X:8, Y:4\}$ .

The 2D mesh maps onto the logical device mesh. TPU v5e pod is  $16 \times 16$  so this slice has NO wraparound links so we can't do a bi-directional AllGather. (Though note, in a mesh each axis doesn't have to map to a specific hardware axis.)

(i)  $E = 2048$ ,  $F = 8192$ , **bf16**

local shape:  $\text{bf16}[\frac{2048}{4}, 8192] = \text{bf16}[512, 8192]$ .

$$T_{\text{total}} = \max\left[T_{\min} \cdot (4 - 1), \frac{2 \cdot 512 \cdot 8192}{4.5 \times 10^{10}} \cdot (4 - 1)\right] = \max[3 \mu\text{s}, 5.59 \times 10^{-4} \text{ s}] = \max[3 \mu\text{s}, 559 \mu\text{s}].$$

This workload is throughput bound:  $T_{\text{total}} = 559 \mu\text{s}$ .

(ii)  $E = 256$ ,  $F = 256$ , **bf16**

local shape:  $\text{bf16}[\frac{256}{4}, 256] = \text{bf16}[64, 256]$ .

$$T_{\text{total}} = \max \left[ 1 \mu\text{s} \cdot (4 - 1), \frac{2 \cdot 64 \cdot 256}{4.5 \times 10} \cdot (4 - 1) \right] = \max[3 \mu\text{s}, 2.18 \mu\text{s}].$$

This load is latency bound:  $T_{\text{total}} = 3 \mu\text{s}$ .

## AllGather and ReduceScatter as derivatives of each other — small example

Let  $n = 2$  (shard size),  $p = 3$  (# of devices),  $u = [1, \dots, 1]^T \in \mathbb{R}^p$ .

Say on forward pass we do an AllGather:

- $x \in \mathbb{R}^{pn}$  (shard of size  $n$  on  $p$  devices).
- $\text{AllGather}(x) : \mathbb{R}^{pn} \rightarrow \mathbb{R}^{p^2 n}$  (all shards on all devices).

$$\begin{aligned} \text{AllGather} = u \otimes I_{pn} &= \begin{bmatrix} I_{pn} \\ \vdots \\ I_{pn} \end{bmatrix} \in \mathbb{R}^{p^2 n \times pn} = \mathbb{R}^{3^2 \cdot 2 \times 3 \cdot 2} = \mathbb{R}^{18 \times 6} \\ &= u \otimes I_n \otimes I_p = \text{broadcast} \otimes I_p \\ &= \begin{bmatrix} I_6 \\ \vdots \\ I_6 \end{bmatrix} \quad p \text{ stacked identity blocks.} \end{aligned}$$

On backward pass: we have  $\partial \ell / \partial y$ ; on forward we did  $y = \text{AllGather}(x)$  (matrix).

$$\Rightarrow \frac{\partial \ell}{\partial x} = (\text{AllGather})^T \frac{\partial \ell}{\partial y}.$$

$$\begin{aligned} (\text{AllGather})^T &= (\text{broadcast} \otimes I_p)^T = \text{reduce} \otimes I_p \\ &= u^T \otimes I_n \otimes I_p \\ &= u^T \otimes I_{pn} \\ &= \text{ReduceScatter}. \end{aligned}$$

$$\Rightarrow \frac{\partial \ell}{\partial x} = \text{ReduceScatter} \frac{\partial \ell}{\partial y}.$$

$$\begin{aligned} \text{ReduceScatter} = u^T \otimes I_{pn} &= [1 \ \cdots \ 1] \otimes I_{pn} = [I_{pn} \ \cdots \ I_{pn}] \\ &= \underbrace{[I_6 \ \cdots \ I_6]}_{p \text{ identity blocks}} \in \mathbb{R}^{6 \times 18}. \end{aligned}$$

## Q1

$A[I_x, J, K, \dots]$ , mesh =  $\{X:4, Y:8, Z:2\}$ . Only sharded along  $X$  and duplicated along  $Y$  and  $Z$ .

Local shape:  $[I/4, J, K, \dots]$ .

$$\text{total bytes taken up by } A \text{ across all chips} = (\# \text{ bytes/elem}) \cdot \underbrace{\frac{\text{sizeof}(A)}{A}}_{A} \cdot A \cdot \underbrace{8 \cdot 2}_{\text{duplications}}$$

$$\text{bytes taken up by 1 copy of } A = (\# \text{ bytes/elem}) \cdot \text{sizeof}(A).$$

Ratio = 16.

## Q2

TPU v4p  $4 \times 4 \times 4$  slice  $\Rightarrow$  full cube so we have wraparound links  $\Rightarrow$  bidirectional ICI b/w.  
mesh =  $\{X:4, Y:4, Z:4\}$ . Assuming ICI link overhead  $T_{\min} = 1 \mu s$ .

(i) **AllGather<sub>x</sub>**( $[B_x, D_y]$ ),  $B = 1024$ ,  $D = 4096$ , **bf16**

$B$  is only sharded along  $X$  so  $Y$  dim comm happens in parallel.

Each device sends a chunk of  $\frac{2 \cdot 1024 \cdot 4096 \cdot 2}{4 \cdot X}$  per hop (bidirectional), over  $\frac{4}{2} = 2$  hops.

$$T_{\text{hop}} = \max\left(T_{\min}, \frac{(1024 \cdot 4096/4)}{9 \times 10^{10}}\right) = \max(1 \mu s, 11.6 \mu s) = 11.6 \mu s.$$

(this op is throughput bound)

$$T_{\text{total}} = T_{\text{hop}} \cdot \frac{4}{2} = 11.6 \mu s \cdot 2 = 23.2 \mu s.$$

(ii) **AllGather<sub>xy</sub>**( $[B_x, D_y]$ ) =  $[B, D]$ ,  $B = 1024$ ,  $D = 4096$ , **bf16**

Multiple axes to gather over.

$$\begin{aligned} T_{\text{total}} &= \max\left[\frac{T_{\min} \cdot 8}{2}, \frac{2 \cdot B \cdot D}{W_{\text{ici}} \cdot 2}\right] \\ &= \max\left[4 \mu s, \frac{2 \cdot 1024 \cdot 4096}{9 \times 10^{10} \cdot 2}\right] \\ &= \max[4 \mu s, 46 \mu s]. \end{aligned}$$

Throughput bound:  $T_{\text{total}} = 46 \mu s$ .

(iii)  $\text{AllReduce}_z([B_x, D_y]\{U_z\}) = [B_x, D_y]$

Can be split up into:

$$\begin{aligned}\text{ReduceScatter}_{z,D}([B_x, D_y]\{U_z\}) &= [B_x, D_{yz}] \\ \text{AllGather}_z([B_x, D_{yz}]) &= [B_x, D_y]\end{aligned}$$

$$T_{\text{total}} = 2 \cdot \frac{BD/(xy)}{W_{\text{ici}}} = 2 \cdot \frac{1024 \cdot 4096 \cdot 2}{4 \cdot 4 \cdot 9 \times 10^{10}} = 11.6 \mu\text{s}.$$

Throughput bound.

### Q3

$\text{AllGather}_x([B_x])$ .  $B$  is very small = 128  $\Rightarrow$  latency bound.

TPU v4p  $4 \times 4 \times 4$  slice, mesh =  $\{X:4, Y:4, Z:4\}$ .

$$T_{\text{total}} = T_{\text{min}} \cdot \frac{4}{2} = 2 \mu\text{s}.$$

For thoroughness:

$$T_{\text{bw}} = \frac{2 \cdot 128}{9\text{e}10} \approx 2.8 \times 10^{-9} \text{ s} \quad (\text{extremely small!})$$

### Q4

$X[B, D] \cdot Y[D_x, F] \rightarrow Z[B, F]$  (assuming bf16 precision).

#### Strategy 1

$\text{AllGather}_x(Y[D_x, F])$  first and then multiply.

$$\begin{aligned}T_{\text{comm}} &= \frac{2 \cdot D \cdot F}{W_{\text{ici}}} \quad (\text{bidirectional}) \\ \text{FLOPs} &= 2BDF \\ T_{\text{comp}} &= \frac{2BDF}{\text{accel FLOPs/s}}\end{aligned}$$

$$\text{Lower bound on } T_{\text{total}} = \max\left(\frac{2DF}{W_{\text{ici}}}, \frac{2BDF}{\text{accel FLOPs/s}}\right).$$

## Strategy 2

Do local sharded matmul  $X[B, D_x] \cdot Y[D_x, F] \rightarrow Z[B, F]\{U_x\}$ , then:

$$\begin{aligned} & \text{AllReduce}_x(Z[B, F]\{U_x\}) \rightarrow Z[B, F] \\ & = \text{ReduceScatter}_{x,F}(Z[B, F]\{U_x\}) \rightarrow Z[B, F_x] \\ & \text{AllGather}_x(Z[B, F_x]) \rightarrow Z[B, F]. \end{aligned}$$

$$\begin{aligned} T_{\text{comm}} &= 2 \cdot \frac{2BF}{W_{\text{ici}}} = \frac{4BF}{W_{\text{ici}}} \\ \text{FLOPs} &= \frac{2BDF}{X}, \quad T_{\text{comp}} = \frac{2BDF/X}{\text{accel FLOPs/s}} \end{aligned}$$

$$\text{Lower bound } T_{\text{total}} = \max\left(\frac{4BF}{W_{\text{ici}}}, \frac{2BDF}{X \cdot \text{accel FLOPs/s}}\right).$$

## Comparing strategies

If our matrix is very large such that the time in Strategy 1 is greater than twice the comm cost, Strategy 2 makes more sense.

To show this (assuming we're on a large model and throughput bound for our comms):

Strat 1 is comm bound when

$$\frac{2DF}{W_{\text{ici}}} > \frac{2BDF}{\text{accel FLOPs/s}} \Rightarrow \frac{\text{accel FLOPs/s}}{W_{\text{ici}}} > B \quad (\text{in bf16}).$$

Using TPU v5p for values going forward:

$$\Rightarrow B < \frac{4.59 \times 10^{14}}{1.8 \times 10^{11}} = 2550.$$

Strat 2 is comm bound when

$$\frac{2 \cdot 4BF}{W_{\text{ici}}} > \frac{2BDF}{X \cdot \text{accel FLOPs/s}} \Rightarrow \frac{2}{W_{\text{ici}}} > \frac{D}{X \cdot \text{accel FLOPs/s}} \Rightarrow \frac{D}{X} < \frac{2 \cdot \text{accel FLOPs/s}}{W_{\text{ici}}} = 5100.$$

Both are typically comms bound for LLM workloads, so comparing  $T_{\text{comm}}$ :

$$\begin{aligned} T_{\text{comm}}^{\text{strat 1}} &= \frac{2DF}{W_{\text{ici}}}, \quad T_{\text{comm}}^{\text{strat 2}} = \frac{4BF}{W_{\text{ici}}}. \\ T_{\text{comm}}^{\text{strat 2}} < T_{\text{comm}}^{\text{strat 1}} &\Rightarrow \frac{4BF}{W_{\text{ici}}} < \frac{2DF}{W_{\text{ici}}} \Rightarrow 2B < D \Rightarrow B < \frac{D}{2}. \end{aligned}$$

Assuming  $D \approx 8000 \Rightarrow B < 4000$ , which is typically true.

For large batch sizes, Strat 1 becomes comp bound while Strat 2 still stays comm bound as it depends on  $X$  being very small, which is almost never true for autoregressive workloads. For Strat 2 to still be better:

$$\begin{aligned} T_{\text{comm}}^{\text{strat 2}} \geq T_{\text{comp}}^{\text{strat 1}} &\Rightarrow \frac{4BF}{W_{\text{ici}}} < \frac{2BDF}{\text{accel FLOPs/s}} \Rightarrow \frac{2}{W_{\text{ici}}} < \frac{D}{\text{accel FLOPs/s}} \\ &\Rightarrow D > \frac{2 \cdot \text{accel FLOPs/s}}{W_{\text{ici}}} = 5100. \end{aligned}$$

For LLMs,  $D \approx 8\text{k}$ , so this is almost always true.

Therefore, for LLM workloads, Strat 2 would almost always be better.

## Q5

(bf16)  $A[I, J] \cdot_J B[J, K] \rightarrow C[I, K]$  on TPU v4p  $4 \times 4 \times 4$ .

Want lowest latency, can start w/ any input sharding but result needs to be fully replicated.

Try different sharding computation cases.

*Note:* # of devices inputs are sharded along do NOT affect  $T_{\text{total}}$  of the comm since the output shape is fixed (assuming we're in bw bound regime).

### Case 1 (Not sharded along contr dim / not fully replicated)

1.  $A[I_{xyz}, J] \cdot B[J, K] \rightarrow C[I_{xyz}, K]$
2.  $\text{AllGather}_{xyz}(C[I_{xyz}, K]) \rightarrow C[I, K]$

$$T_{\text{comm}} = \frac{2 \cdot IK}{W_{\text{ici}} \cdot 3} = \frac{2IK}{3 \cdot 9 \times 10^{10}}.$$

How are sharded doesn't really matter since the AllGather cost will stay about the same.

$$\text{FLOPS per TPU} = 2 \left( \frac{I}{xyz} \right) \cdot J \cdot K = \frac{2JKI}{xyz} = \frac{2JKI}{64} = \frac{JKI}{32}.$$

Regardless of how we split up the sharding, FLOPS/TPU stay the same (unless we duplicate along some hardware axis, but this would be suboptimal as we'll just have more FLOPS/TPU to do).

### Case 2 (Fully duplicated on each device)

$$A[I, J] \cdot B[J, K] \rightarrow C[I, K].$$

No comm cost, but very large matrices may not be able to fit on a single TPU.

$$T_{\text{comp}} = \frac{2IJK}{\text{accel FLOPs/s}} = \frac{2IJK}{2.75 \times 10^{14}}.$$

### Case 3 (one operand has a sharded contr dim)

$$A[I, J_{xyz}] \cdot B[J, K] \rightarrow C[I, K].$$

This case is suboptimal to Case 1 no matter what kind of sharding variation you try since you have to AllGather at the start and will have more FLOPS/TPU to do along with potentially having to do a final AllGather.

### Case 4 (both operands are sharded along contr dim)

$$A[I, J_{xyz}] \cdot B[J_{xyz}, K] \rightarrow C[I, K]\{U_{xyz}\}.$$

$$\text{AllReduce}_{xyz}(C[I, K]\{U_{xyz}\}) \rightarrow C[I, K].$$

Any other sharding pattern would be suboptimal since we'd have to do either more FLOPS/TPU or an extra AllGather at the end.

$$T_{\text{comp}} = \frac{2IJK/(xyz)}{\text{accel FLOPs/s}}, \quad T_{\text{comm}} = \frac{2 \cdot 2IK}{W_{\text{ici}} \cdot 3}.$$

### Comparison

$$\begin{aligned} T_{\text{total}}^{\text{case 1}} &= \max \left\{ \frac{2IK}{3 \cdot 9 \times 10^{10}}, \frac{IJK/32}{2.75 \times 10^{14}} \right\} \\ T_{\text{total}}^{\text{case 2}} &= \frac{2IJK}{2.75 \times 10^{14}} \\ T_{\text{total}}^{\text{case 4}} &= \max \left\{ \frac{4IK}{3 \cdot 9 \times 10^{10}}, \frac{IJK/64}{2.75 \times 10^{14}} \right\} \end{aligned}$$

Case 4 has the same FLOPS/chip as Case 1 but double the comm cost. Case 2 has no comm cost but in most LLM cases may not even be feasible.

⇒ **Lowest latency sharding:**

1.  $A[I_{xyz}, J] \cdot B[J, K] \rightarrow C[I_{xyz}, K]$
2.  $\text{AllGather}_{xyz}(C[I_{xyz}, K]) \rightarrow C[I, K]$

$$T_{\text{total}} = \max \left\{ \frac{2IK}{3 \cdot 9 \times 10^{10}}, \frac{IJK/32}{2.75 \times 10^{14}} \right\}.$$

### Q6

$4 \times 4$  v5e slice has no wraparound connection ⇒ can't use bidirectional ICI.

(i)  $A[I_x, J_y] \cdot B[J_y, K] \rightarrow C[I_x, K]$  **on TPU v5e**  $4 \times 4$

$$\begin{aligned} &A[I_x, J_y] \cdot_{\text{LOCAL}} B[J_y, K] \rightarrow C[I_x, K]\{U_y\} \\ &\text{AllReduce}_y(C[I_x, K]\{U_y\}) \rightarrow C[I_x, K] \end{aligned}$$

$$T_{\text{comp}} = \frac{2 \cdot (I/X) \cdot (J/Y) \cdot K}{\text{v5e FLOPs/s}} = \frac{2IJK}{XY \cdot \text{v5e FLOPs/s}} = \frac{2IJK}{16 \cdot 1.97\text{e}14} \quad (\text{assuming bf16}).$$

$$T_{\text{comm}} = 2 \cdot \frac{(IK \cdot 2/X) \cdot (Y-1)}{Y \cdot W_{\text{ici}}} \quad (\text{unidirectional}) = \frac{4IK(Y-1)}{XYW_{\text{ici}}} = \frac{12IK}{16 \cdot 4.5\text{e}10}.$$

$$\text{(ii)} \quad A[I_x, J] \cdot_J B[J_x, K_y] \rightarrow C[I_x, K_y]$$

$$\begin{aligned} \text{AllGather}_x(B[J_x, K_y]) &\rightarrow B[J, K_y] \\ A[I_x, J] \cdot_J B[J, K_y] &\rightarrow C[I_x, K_y] \end{aligned}$$

$$\begin{aligned} T_{\text{comp}} &= \frac{2 \cdot (I/X) \cdot J \cdot (K/Y)}{\text{v5e FLOPs/s}} = \frac{2IJK}{XY \cdot \text{v5e FLOPs/s}} = \frac{2IJK}{16 \cdot 1.97\text{e}14}. \\ T_{\text{comm}} &= \frac{2J \cdot (K/Y)}{X \cdot \underbrace{W_{\text{ici}}}_{\text{uni}}} \cdot (X-1) = \frac{2JK(X-1)}{YX \cdot W_{\text{ici}}} = \frac{6JK}{16 \cdot 4.5\text{e}10}. \end{aligned}$$

$$\text{(iii)} \quad A[I_x, J] \cdot_J B[J, K_y] \rightarrow C[I_x, K_y]$$

Can do comp locally, no comm required.

$$T_{\text{comp}} = \frac{2 \cdot (I/X) \cdot J \cdot (K/Y)}{\text{v5e FLOPs/s}} = \frac{2IJK}{16 \cdot 1.97\text{e}14}, \quad T_{\text{comm}} = 0.$$

## Q7

Transformer block ( $F \gg D$ ):

$$\text{In}[B, D] \cdot W_{\text{in}}[D, F] \cdot W_{\text{out}}[F, D].$$

$D = 8192$ ,  $F = 32768$ ,  $B = 128$ , bf16, TPU v5e  $2 \times 2$  (no wraparound / bidir ici). Each TPU has only  $300 \text{ MB} = 3 \times 10^8$  bytes of free memory.

$$\begin{aligned} \text{sizeof(In)} &= 2 \cdot B \cdot D = 2 \cdot 128 \cdot 8192 = 2,097,152 \approx 2 \text{ MB} \\ \text{sizeof}(W_{\text{in}}) &= 2 \cdot D \cdot F = 2 \cdot 8192 \cdot 32768 \approx 536 \text{ MB} \\ \text{sizeof}(W_{\text{out}}) &= 2 \cdot F \cdot D \approx 536 \text{ MB} \end{aligned}$$

$$\text{sizeof}(W_{\text{in}}[D, F_{xy}]), \text{ sizeof}(W_{\text{out}}[F_{xy}, D]) = 131.5 \text{ MB}.$$

Intermediate output also needs to fit in TPU memory.



## Try 1

$$\begin{aligned}\text{In}[B, D] \cdot W_{\text{in}}[D, F_{xy}] \cdot W_{\text{out}}[F_{xy}, D] &= \text{Tmp}[B, F_{xy}] \cdot W_{\text{out}}[F_{xy}, D] \\ &= \text{Out}[B, D]\{U_{xy}\}\end{aligned}$$

$$\text{AllReduce}_{xy}(\text{Out}[B, D]\{U_{xy}\}) = \text{Out}[B, D].$$

Memory used per TPU:

$$\begin{aligned}&\text{sizeof}(\text{In}[B, D]) + \text{sizeof}(W_{\text{in}}[D, F_{xy}]) + \text{sizeof}(W_{\text{out}}[F_{xy}, D]) \\ &\quad + \underbrace{\text{sizeof}(\text{Tmp}[B, F_{xy}])}_{2 \cdot 128 \cdot 32768 / 4 = 2 \text{ MB}} + \underbrace{\text{sizeof}(\text{Out}[B, D])}_{2 \cdot 128 \cdot 8192 = 2 \text{ MB}} \\ &= 2 \text{ MB} + 131.5 \text{ MB} + 131.5 \text{ MB} + 2 \text{ MB} + 2 \text{ MB} = 269 \text{ MB} \checkmark \text{ fits.}\end{aligned}$$

$$\begin{aligned}T_{\text{comp}} &= \frac{(2 \cdot BD \cdot F / (XY)) + (2 \cdot B \cdot F / (XY) \cdot D)}{\text{v5e FLOPs/s}} = \frac{2BDF}{XY \cdot \text{v5e FLOPs/s}} \\ &= \frac{2 \cdot 128 \cdot 8192 \cdot 32768}{4 \cdot 1.97\text{e}14} = 87 \mu\text{s} \\ &\quad (\text{time for each matmul is } 43.5 \mu\text{s}).\end{aligned}$$

$$\begin{aligned}T_{\text{comm}} &= 2 \cdot \frac{(2 \cdot BD) \cdot (X + Y - 1)}{XY \cdot 2 \cdot \underbrace{W_{\text{ici}}}_{\text{unidir}}} = \frac{2BD(X + Y - 1)}{XYW_{\text{ici}}} \\ &= \frac{2 \cdot 128 \cdot 8192 \cdot (4 - 1)}{4 \cdot 4.5 \times 10^{10}} \approx 34.9 \mu\text{s}.\end{aligned}$$

In this case, the operation is comp bounded so the AR can be masked by the 2nd matmul. Therefore, the time under this sharding scheme gives the same latency as sharding  $\text{In}[B, D_{xy}]$  and  $\text{Out}[B, D_{xy}]$  even though full duplication may initially seem suboptimal.

## Q8

### Code

```
1 import jax
2 import jax.numpy as jnp
3
4 from jax.sharding import Mesh, PartitionSpec as P
5
6 """
7 Allocate a 2d matrix and shard
8 across a 2d hardware grid.
9
10 Compare benchmarked time to ideal comm time.
11 """
```

```

12
13 assert len(jax.devices()) == 8
14 mesh = jax.make_mesh(axis_shapes=(4, 2), axis_names=('x', 'y'))
15 jax.set_mesh(mesh)
16
17 print("Benchmark AllGather and compare to theoretical time.")
18
19 """
20 Shard our array A so that each
21 device has 1 element.
22
23 A[I_x, J_y]
24 local shape: [1000, 100]
25 """
26 A = jax.device_put(jnp.arange(4 * 1000 * 2 * 100, dtype="bfloat16").reshape(4 *
    1000, 2 * 100), P('x', 'y'))
27 print("Original array sharding:")
28 jax.debug.visualize_array_sharding(A)
29
30
31 """
32 Note: v5e-8 slice has no wraparound connection, so assume W_ici = unidirectional
    bandwidth.
33
34 AllGather_x(A[I_x, J_y]) = A[I, Jy]
35
36 Latency-bound:
37 Assuming T_min = 1us
38
39 T_total = T_min * (x - 1) = 3us.
40
41 Bandwidth-bound:
42
43 T_total = 2 * (I / x) * (J / y) * (x - 1) / W_ici
44 = 2 * 4 * 1000 / 4 * 2 * 100 / 2 * 3 / 4.5e10
45 = 13us
46
47 This load is bandwidth-bound and we should expect it to take around 13us.
48 """
49
50 @jax.shard_map(in_specs=(P('x', 'y')), out_specs=P('x', 'y'))
51 def f1(x_block):
52     return jax.lax.all_gather(x_block, 'x', tiled=True)
53
54 print("Above all gather function time:")
55 f1_jit = jax.jit(f1)
56 f1_jit(A).block_until_ready() # initial jit compilation
57 %timeit f1_jit(A).block_until_ready() # measure actual time-ish (probably still
    including some other overhead as well)
58
59
60
61
62
63
64 print("\n\nBenchmark AllReduce and compare to theoretical time.")
65
66 """
67 Shard our array A so that rows are sharded along x

```

```

68 and duplicated along y.
69
70 A[I_x, J]
71 local shape: [1000, 100]
72 """
73 A = jax.device_put(jnp.arange(4 * 1000 * 2 * 100, dtype="bfloat16").reshape(4 *
    1000, 2 * 100), P('x', None))
74 print("Original array sharding:")
75 jax.debug.visualize_array_sharding(A)
76
77
78 """
79 AllReduce_y(A[I_x, J]{U_y}) = A[I_x, J]
80
81 Example decomposition into AG and RS:
82 ReduceScatter_yJ(A[I_x, J]{U_y}) = A[I_x, J_y]
83 AllGather_y(A[I_x, J_y]) = A[I_x, J]
84
85 Latency-bound:
86 T_total = 2 * 1us * (y - 1) = 2us
87
88 Bandwidth-bound:
89 T_total = 2 * (2 * (I/x) * (J/y)) * (y - 1) / W_ici
90 = 2 * 2 * (4000 / 4) * (200 / 2) * 1 / 9e10
91 = 4000 * 400 / 9e10 = 17us
92
93 Above is bandwidth bound.
94 """
95 @jax.shard_map(in_specs=(P('x', None)), out_specs=P('x', None))
96 def f2(x_block):
97     return jax.lax.psum(x_block, 'y')
98
99 print("Above all reduce function time")
100 f2_jit = jax.jit(f2)
101 f2_jit(A).block_until_ready()
102 %timeit f2_jit(A).block_until_ready()
103
104
105
106
107
108
109 print("\n\nBenchmark ReduceScatter and compare to theoretical time.")
110
111 """
112 Shard A rows along x and duplicate along y.
113
114 A[I_x, J]
115 local shape: [4 * 1000 / 4, 2 * 100]
116 """
117 A = jax.device_put(jnp.arange(4 * 1000 * 2 * 100, dtype="bfloat16").reshape(4 *
    1000, 2 * 100), P('x', None))
118 print("Original array sharding:")
119 jax.debug.visualize_array_sharding(A)
120
121 """
122 ReduceScatter_yJ(A[I_x, J]{U_y}) = A[I_x, J_y]
123
124 Latency-bound:

```

```

125 T_total = 1us * (y-1) = 1us
126
127 Bandwidth-bound:
128 T_total = 2 * (I / x) * (J / y) * (y - 1) / W_ici
129 = 2 * 4000 / 4 * 200 / 2 * 1 / 4.5e10
130 ~= 4us
131
132 The above comm is bandwidth bounded.
133 """
134 @jax.shard_map(in_specs=(P('x', None)), out_specs=P('x', 'y'))
135 def f3(x_block):
136     return jax.lax.psum_scatter(x_block, 'y', scatter_dimension=1, tiled=True)
137
138 print("Above reduce scatter function time")
139 f3_jit = jax.jit(f3)
140 f3_jit(A).block_until_ready()
141 %timeit f3_jit(A).block_until_ready()
142
143
144
145
146
147
148 print("\n\nBenchmark AllToAll and compare to theoretical time.")
149
150 """
151 Shard A rows along x and duplicate along y.
152
153 A[I_x, J]
154 local shape: [4 * 1000 / 4, 2 * 100]
155 """
156 # A = jax.device_put(jnp.arange(4 * 1000 * 2 * 100, dtype="bfloat16").reshape(4
157 * 1000, 2 * 100), P('x', None))
158 A = jax.device_put(jnp.arange(4 * 4, dtype="bfloat16").reshape(4, 4), P('x',
159 None))
160
161 print("Original array sharding:")
162 jax.debug.visualize_array_sharding(A)
163
164 """
165 AllToAll_xJ(A[I_x, J]) = A[I, J_x]
166
167
168 Latency-bound:
169 T_total = 1us * (y-1) = 1us
170
171
172 Bandwidth-bound:
173 T_total = (1/4) * 2 * (I / x) * J * (x - 1) / W_ici
174 = 1/2 * 4000 / 4 * 200 * 3 / 4.5e10 = 6us
175 """
176
177 @jax.shard_map(in_specs=(P('x', None)), out_specs=P(None, 'x'))
178 def f4(x_block):
179     return jax.lax.all_to_all(x_block, 'x', split_axis=1, concat_axis=0, tiled=
180 True)
181
182 print("Above all to all function time")
183 f4_jit = jax.jit(f4)
184 f4_jit(A).block_until_ready()
185 %timeit f4_jit(A).block_until_ready()

```

## Results



## Q9

1. Step 1: On each device, do a local matmul.

$$A[N, M_x] \cdot_{\text{LOCAL}} B[M_x, K] = C[N, K]\{U_x\}.$$

- Step 2: Do an AllReduce along the  $x$ -axis to sum up the partial sums.

$$\text{AllReduce}_x(C[N, K]\{U_x\}) = C[N, K].$$

2. The comm primitive we choose changes.

$$\text{Step 1: } A[N, M_x] \cdot_{\text{LOCAL}} B[M_x, K] = C[N, K]\{U_x\}.$$

$$\text{Step 2: } \text{ReduceScatter}_{x,K}(C[N, K]\{U_x\}) = C[N, K_x].$$

3. The comm cost w/ AG:

$$\text{AllGather}_x(A[N, M_x]) = A[N, M], \quad T_{\text{total}} = \frac{2 \cdot N \cdot M}{W_{\text{ici}}}.$$

Comm cost w/ RS:

$$\text{ReduceScatter}_{x,K}(C[N, K]\{U_x\}) = C[N, K_x], \quad T_{\text{total}} = \frac{2NK}{W_{\text{ici}}}.$$

$$\frac{T_{\text{total}}^{\text{AG}}}{T_{\text{total}}^{\text{RS}}} = \frac{2NM/W_{\text{ici}}}{2NK/W_{\text{ici}}} = \frac{M}{K}.$$

## Q10

1. On a link entering any device, it receives a shard from every other device for AG/RS.

$$\Rightarrow \frac{N^2}{D} \cdot (D-1) \text{ total scalars transferred} = N^2 \cdot \left(1 - \frac{1}{D}\right) \approx N^2 \text{ as } D \gg N.$$

2. Each device  $i$  has shard  $[A_i, B_i, C_i, D_i]$ . Afterwards device  $0, \dots$  has shards like  $[A_0; \dots; A_3]$ .

Focusing into the highlighted link ( $3 \rightarrow 0$ ):

- dev 1 sends dev 0:  $A_1$
- dev 2 sends dev 0:  $A_2$
- dev 2 sends dev 1:  $B_2$
- dev 3 sends dev 0:  $A_3$
- dev 3 sends dev 1:  $B_3$
- dev 3 sends dev 2:  $C_3$

Generalizing: in current 1D topology, every link follows this pattern where all devices traverse that link  $X$  times depending on how many devices come before it and the links end.

$$\Rightarrow (1+2+\dots+(D-1)) \cdot \frac{N^2}{D^2} = \frac{(D-1)D}{2} \cdot \frac{N^2}{D^2} = \frac{N^2(D-1)}{2D} = \frac{N^2}{2} \left(1 - \frac{1}{D}\right) \approx \frac{N^2}{2} \text{ as } D \gg N.$$

- 3.

$$\frac{\text{Data}_{\text{AllToAll}}}{\text{Data}_{\text{AG/RS}}} = \frac{1}{2}.$$

In an AG/RS, each device from its receiving link receives  $(D-1)$  full shards =  $(D-1) \cdot D$  smaller shards =  $((D-1) + (D-1) + \dots + (D-1))$  smaller shards.

For an AllToAll, we're sending across each link  $(1 + 2 + \dots + (D-1))$  smaller shards.

Expanding it out, we simply transfer less data across devices since the end/start state of an AG/RS is each device having a tensor whose size is equal to the full array size, whereas for AllToAll, the before and after device tensor size is the same.

4. The time halves since the data needs to travel less hops / ici links.

5. With bidirectional links, every device before a link only uses the link to reach  $\lfloor D/2 \rfloor$  devices after the links end.

$\Rightarrow$  Across each link we now send

$$\begin{aligned}
\left( \frac{D}{2} + \left( \frac{D}{2} - 1 \right) + \dots + 1 \right) \cdot \frac{N^2}{D^2} &= \frac{(D/2)(D/2 + 1)}{2} \cdot \frac{N^2}{D^2} \\
&= \frac{D^2/4 + D/2}{2} \cdot \frac{N^2}{D^2} = \left( \frac{D^2}{8} + \frac{D}{4} \right) \cdot \frac{N^2}{D^2} \\
&= \frac{D^2 + 2D}{8} \cdot \frac{N^2}{D^2} = \frac{1 + (2/D)}{8} \cdot N^2 \\
&= \frac{1}{4} \cdot \frac{N^2}{2} \left( 1 + \frac{2}{D} \right), \quad \text{as } D \gg N \\
&\Rightarrow \frac{N^2}{8} \quad (4 \text{ times lower time}).
\end{aligned}$$

- 6.

$$\frac{T^{\text{AG,bidir}}}{T^{\text{AllToAll,bidir}}} = \frac{N^2/2}{N^2/8} = \frac{1}{2} \cdot 8 = 4.$$

As mentioned in (3), the speedup happens b/c we're not sending the whole shard from one device to other devices each time. When we add bidirectional links to our AllToAll, we get to send less data across each link along with taking less hops. Our original AllToAll (unidir) was  $2\times$  the unidir AG. Adding bidir links  $2\times$  the speed of the AG but  $4\times$  the AllToAll, hence the ratio of 4.